

[On This Page](#) >

# Clan service

This document describes the structure and configurable attributes of a `clan.service` module.

Typically needed by module authors to define roles, behavior and metadata for distributed services.

## Note

This is not a user-facing documentation, but rather meant as a reference for *module authors*

See: [clanService Authoring Guide](#)

## "#specialArgs"

Attribute: `"#specialArgs"`

Additional arguments passed to the module. Often referred to as `specialArgs`.

```
1 {
2   # root directory of the clan
3   directory,
4   # clanLib - The clan library functions
5   clanLib,
6   # exports from all services
7   exports,
8   ...
9 }:
10 {
11   _class = "clan.service";
12   manifest.name = "example-service";
```

```

13
14 # ... elided
15 }

```

Type: unspecified value

## exports

Attribute: exports

### ❖ Experimental Feature

This feature is experimental and will change in the future.

This services exports. Gets merged with all other services exports.

Exports are used to share and expose information between instances.

Define exports in the `perInstance` or `perMachine` scope.

Accessing the exports:

```

1 { exports, ... }: {
2   _class = "clan.service";
3
4   # ...
5   roles.peer.perInstance = { exports, ... }: { ... };
6
7   # ...
8   perMachine = { exports, ... }: { ... };
9 }

```

Type: lazy attribute set of module

Default:

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](https://clan.lol/docs/25.11/reference/options/clan_service)

# manifest

---

**Attribute:** `manifest`

Meta information about this module itself

**Type:** `submodule`

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## categories

**Attribute:** `manifest.categories`

Categories are used for Grouping and searching.

While initial oriented on [freedesktop](#) the following categories are allowed

**Type:** `list of (one of "Audio", "AudioVideo", "Desktop", "Development", "Education", "Game", "Graphics", "Network", "Office", "Science", "Settings", "Social", "System", "Uncategorized", "Utility", "Video")`

**Default:**

```
[  
  "Uncategorized"  
]
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## description

**Attribute:** `manifest.description`

A Short description of the module.

**Type:** `string`

**Default:**

"No description"

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## features

**Attribute:** `manifest.features`

Enable built-in features for the module

See the documentation for each feature:

- API

**Type:** `submodule`

**Default:**

`{ }`

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## API

**Attribute:** `manifest.features.API`

**Readonly**

Enables automatic API schema conversion for the interface of this module.

**Type:** `boolean`

**Default:**

`true`

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

# name

**Attribute:** `manifest.name`

The name of the module

Mainly used to create an error context while evaluating. This helps backtracking which module was included; And where an error came from originally.

**Type:** `string`

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

# readme

**Attribute:** `manifest.readme`

Extended usage description

**Type:** `string`

**Default:**

""

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

# perMachine

---

**Attribute:** `perMachine`

Per-machine configuration of the service.

This option is used to define machine-specific settings for the service **once**, if any service-instance is used.

Although the type is a `deferredModule`, it helps to think of it as a function. The 'function' takes the `machine-name` and some other 'arguments'

*Arguments:*

- `machine`: `{ name :: string; roles :: listOf String }`
- `instances`: The scope of the machine, containing all instances and roles that the machine is part of.

```

1 {
2   instances = {
3     <instanceName> = {
4       roles = {
5         <roleName> = {
6           # Resolved per-machine settings
7           machines = { <machineName> = { settings = { ... }; }; };
8         };
9       };
10    };
11 }

```

Returns an `attribute set` containing:

- `nixosModule`: The NixOS module for the machine.

**Type:** `module`

**Default:**

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## darwinModule

**Attribute:** `perMachine.darwinModule`

A single nix-darwin module for the machine.

This module is later imported to configure darwin machines with the config derived from service's settings.

Example:

```

1 #           ↓ machine.roles ...
2 perMachine = { machine, ... }:

```

```

3 {                                # ↓ darwin-config
4   darwinModule = { config ,... }: {
5     launchd.daemons.foo = {
6       enable = true;
7     };
8   }
9 }

```

Type: `module`

Default:

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## exports

Attribute: `perMachine.exports`

### ❖ Experimental Feature

This feature is experimental and will change in the future.

export modules defined in 'perMachine' mapped to their machine name

Example

with machines:

```

1 instances.A = { roles.peer.machines.jon = ... };
2 instances.B = { roles.peer.machines.jon = ... };
3
4 perMachine = { machine, ... }:
5 {
6   exports.foo = 1;
7 }
8
9 This yields all other services can access these exports
10 =>
11 exports.machines.jon.foo = 1;

```

```
12 exports.machines.sara.foo = 1;
```

**Type:** lazy attribute set of module

**Default:**

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## nixosModule

**Attribute:** perMachine.nixosModule

A single NixOS module for the machine.

This module is later imported to configure the machine with the config derived from service's settings.

Example:

```
1 #           ↓ machine.roles ...
2 perMachine = { machine, ... }:
3 {           # ↓ nixos-config
4   nixosModule = { config, ... }: {
5     systemd.services.foo = {
6       enable = true;
7     };
8   }
9 }
```

**Type:** module

**Default:**

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)



# roles

**Attribute:** `roles`

Roles of the service.

A role is a specific behavior or configuration of the service. It defines how the service should behave in the context of the clan.

The `<roleName>`s of the service are defined here. Later usage of the roles must match one of the `roleNames`.

For example:

- `'roles.client = ...'` for a client role that connects to the service
- `'roles.server = ...'` for a server role that provides the service

Throws an error if empty, since this would mean that the service has no way of adding members.

**Type:** `attribute set of (submodule)`

**Default:**

"Throws: 'The service must define its roles' when not defined"

Declared in: [lib/inventory/distributed-service/service-module.nix](https://clan.lol/docs/25.11/reference/options/clan_service)

## description

**Attribute:** `roles.<roleName>.description`

A short description of the role '`name`', explaining it's effect on the supplied machine.

**Type:** `null or string`

**Default:**

`null`

◆ Example



"Connects the supplied machine as a '<name>' to the 'example' service."

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## interface

**Attribute:** `roles.<roleName>.interface`

Abstract interface of the role.

This is an abstract module which should define 'options' for the role's settings.

Example:

```
1 {
2   options.timeout = mkOption {
3     type = types.int;
4     default = 30;
5     description = "Timeout in seconds";
6   };
7 }
```

Note:

- `machine.config` is not available here, since the role is definition is abstract.
- *defaults* that depend on the *machine* or *instance* should be added to *settings* later in 'perInstance' or 'perMachine'

**Type:** `module`

**Default:**

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## perInstance

**Attribute:** `roles.<roleName>.perInstance`

Per-instance configuration of the role.

This option is used to define instance-specific behavior for the service-role. (Example below)

Although the type is a `deferredModule`, it helps to think of it as a function. The 'function' takes the `instance-name` and some other `arguments`.

*Arguments:*

- `instanceName` (`string`): The name of the instance.
- `machine`: Machine information, containing:

```

1  {
2    name = "machineName";
3    roles = ["client" "server" ... ];
4  }
```

- `roles`: Attribute set of all roles of the instance, in the form:

```

1  roles = {
2    client = {
3      machines = {
4        jon = {
5          settings = {
6            timeout = 60;
7          };
8        };
9        # ...
10     };
11   };
12   # ...
13 };
```

- `settings`: The settings of the current machine, as defined in `instances`

```

{
  timeout = 30;
}
```

- `extendSettings`: A function that takes a module and returns a new module with extended settings.

```

1 extendSettings {
2   timeout = mkForce 60;
3 };
4 ->
5 {
6   timeout = 60;
7 }

```

Returns an `attribute set` containing:

- `nixosModule`: The NixOS module for the instance.

Type: `module`

Default:

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## darwinModule

Attribute: `roles.<roleName>.perInstance.darwinModule`

This module is later imported to configure darwin machines with the config derived from service's settings.

Example:

```

1 roles.client.perInstance = { instanceName, ... }:
2 {
3   # Keep in mind that this module is produced once per-instance
4   # Meaning you might end up with multiple of these modules.
5   # Make sure they can be imported all together without conflicts
6   #
7   #           ↓ darwin-config
8   darwinModule = { config, ... }: {
9     # create one launchd daemon per instance
10    # It is a common practice to concatenate the *service-name* and *inst
11    # To ensure globally unique launchd daemons for the target machine
12    launchd.daemons."webly-${instanceName}" = {
13      ...
14    };

```

```
15   };
16 }
```

**Type:** `module`

**Default:**

```
{ }
```

Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## exports

**Attribute:** `roles.<roleName>.perInstance.exports`

### ❖ Experimental Feature

This feature is experimental and will change in the future.

export modules defined in 'perInstance' mapped to their instance name

Example

with instances:

```
1 instances.A = { ... };
2 instances.B = { ... };
3
4 roles.peer.perInstance = { instanceName, machine, ... }:
5 {
6   exports.foo = 1;
7 }
8
9 This yields all other services can access these exports
10 =>
11 exports.instances.A.foo = 1;
12 exports.instances.B.foo = 1;
```

**Type:** `lazy attribute set of module`

**Default:**

`{ }`Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## nixosModule

Attribute: `roles.<roleName>.perInstance.nixosModule`

This module is later imported to configure the machine with the config derived from service's settings.

Example:

```

1 roles.client.perInstance = { instanceName, ... }:
2 {
3   # Keep in mind that this module is produced once per-instance
4   # Meaning you might end up with multiple of these modules.
5   # Make sure they can be imported all together without conflicts
6   #
7   #                               ↓ nixos-config
8   nixosModule = { config, ... }: {
9     # create one systemd service per instance
10    # It is a common practice to concatenate the *service-name* and *inst
11    # To ensure globally unique systemd-units for the target machine
12    systemd.services."webly-${instanceName}" = {
13      ...
14    };
15  };
16 }

```

Type: `module`

Default:

`{ }`Declared in: [lib/inventory/distributed-service/service-module.nix](#)

## Inventory Submodule

Next →  
clan.core

Need help? Reach out to [our community](#).